

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Práctica: CRUD de Carrera con JDBC, el patrón DAO y procedimientos almacenados (CallableStatement)

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

19 de septiembre de 2026

Índice

1. Introducción	2
2. Desarrollo de la aplicación	2
2.1. La tabla en la base de datos	2
2.2. Las clases DTO y DAO	2
2.3. Los procedimientos almacenados	2
2.4. Los métodos CRUD con CallableStatement	4
2.5. La interfaz gráfica y el CRUD completo	5
2.6. Evidencia de la conexión JDBC y de las llamadas a los procedimientos	9
3. Conclusiones	10
4. Referencias Bibliográficas	11

Índice de figuras

1.	Estructura de la tabla Carrera (<code>describe Carrera;</code>)	2
2.	Los cinco procedimientos almacenados creados en la base MisCursos	4
3.	Ejecución directa de dos procedimientos desde el cliente de MySQL	4
4.	Estado inicial: solo Nuevo y Listar habilitados, tabla vacía	5
5.	Alta: formulario lleno tras presionar Nuevo, con Guardar y Cancelar habilitados	6
6.	Primera carrera guardada y leída de vuelta desde la base	6
7.	Segunda alta: dos registros para probar cambio, baja y error	7
8.	Error al repetir un nombre: restricción UNIQUE de nombreCarrera	7
9.	Carrera 1 seleccionada, con la descripción editada antes de Actualizar	8
10.	Carrera 1 con la descripción actualizada	8
11.	Confirmación antes de eliminar la carrera 2	9
12.	Estado final: solo queda la carrera 1, ya actualizada	9
13.	Registro del servidor: conexiones JDBC del DAO y llamadas a los procedimientos	10
14.	Contenido real de la tabla tras las operaciones hechas desde la ventana	10

Índice de tablas

1. Correspondencia entre las operaciones del CRUD, los métodos del DAO y los procedimientos . . . 3
2. Errores típicos al trabajar con JDBC, DAO y procedimientos almacenados 11

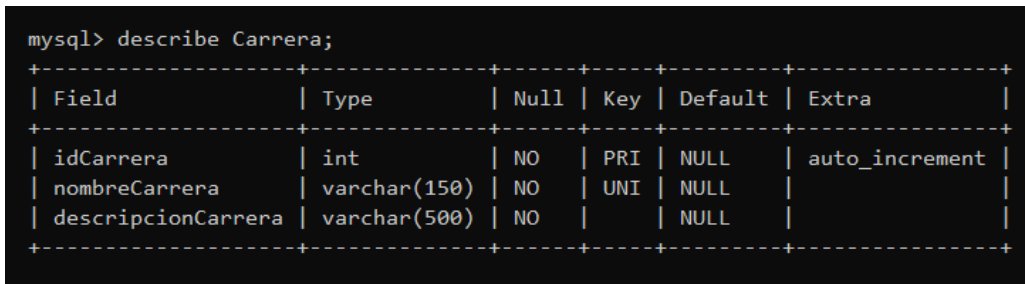
1. Introducción

Esta práctica conecta una aplicación de escritorio en Java con MySQL a través de JDBC. El acceso a datos sigue el patrón DAO y el SQL vive en la base como cinco procedimientos almacenados (*stored procedures*), que `CarreraDAO` invoca con `CallableStatement`. La ventana Swing (`VentanaCarrera`) solo llama a los cinco métodos del DAO.

2. Desarrollo de la aplicación

2.1. La tabla en la base de datos

La base `MisCursos` corre en un contenedor de MySQL 8.0 (`docker-compose.yml` del proyecto), publicado en el puerto 3307 del equipo. Al crearse el contenedor por primera vez se ejecutan, en orden, los dos scripts de la carpeta `init/`: `01-schema.sql` crea la tabla y `02-procedimientos.sql` crea los procedimientos. La figura 1 muestra la estructura de la tabla con `describe Carrera;`. El campo `nombreCarrera` es `UNIQUE`, lo que más adelante produce el error de la figura 8.



Field	Type	Null	Key	Default	Extra
idCarrera	int	NO	PRI	NULL	auto_increment
nombreCarrera	varchar(150)	NO	UNI	NULL	
descripcionCarrera	varchar(500)	NO		NULL	

Figura 1: Estructura de la tabla Carrera (`describe Carrera;`)

2.2. Las clases DTO y DAO

DTO (`Carrera`). Es un objeto de transferencia de datos: solo guarda los tres atributos de la tabla (`idCarrera`, `nombreCarrera` y `descripcionCarrera`), todos privados, con sus *getters* y *setters* y un `toString`. No contiene lógica ni sabe nada de SQL. Es lo que viaja entre la ventana y el DAO.

DAO (`CarreraDAO`). Concentra todo el acceso a la base de datos [1]. Expone cinco métodos públicos (`create`, `read`, `readAll`, `update` y `delete`) y un método privado, `obtenerConexion()`, que carga el driver y abre la conexión JDBC. La ventana solo importa `SQLException` de `java.sql`, para atrapar los errores que declara el DAO y mostrarlos en un cuadro de diálogo. No abre conexiones ni arma consultas.

Código 1: Conexión JDBC en `CarreraDAO` (`obtenerConexion`)

```
1 private void obtenerConexion() throws SQLException {
2     String usuario = "root";
3     String clave = "tu_password";
4     String urlBD = "jdbc:mysql://localhost:3307/MisCursos";
5     String driverMySQL = "com.mysql.cj.jdbc.Driver";
6
7     try {
8         Class.forName(driverMySQL);
9     } catch (ClassNotFoundException e) {
10         throw new SQLException("No se encontro el driver de MySQL: "
11                                 + driverMySQL, e);
12     }
13     conexion = DriverManager.getConnection(urlBD, usuario, clave);
14 }
```

La URL apunta al puerto 3307 y a la base `MisCursos`. El driver es el conector de MySQL (`Connector/J`) [2], que debe estar en el *classpath* al compilar y ejecutar. Si no lo está, `ClassNotFoundException` se convierte en `SQLException`, de modo que todo el DAO declara un único tipo de error.

2.3. Los procedimientos almacenados

Un procedimiento almacenado es un bloque de SQL guardado en el servidor con un nombre y parámetros, que se ejecuta con `CALL` [3]. Se creó uno por cada operación del CRUD (código 2). Todos los parámetros son de

entrada (IN). Como el cuerpo de cada procedimiento contiene puntos y coma, el script cambia temporalmente el delimitador a \$\$ con DELIMITER; si no, el cliente cortaría la sentencia en el primer ;.

Código 2: Los cinco procedimientos almacenados (init/02-procedimientos.sql)

```

1  — Los 5 stored procedures del CRUD de Carrera , uno por cada metodo del DAO.
2  DELIMITER $$
3
4  CREATE PROCEDURE sp_crear_carrera(
5      IN p_nombre VARCHAR(150),
6      IN p_descripcion VARCHAR(500)
7  )
8  BEGIN
9      INSERT INTO Carrera (nombreCarrera , descripcionCarrera)
10     VALUES (p_nombre, p_descripcion);
11 END$$
12
13 CREATE PROCEDURE sp_leer_carrera(
14     IN p_id INT
15 )
16 BEGIN
17     SELECT idCarrera , nombreCarrera , descripcionCarrera
18     FROM Carrera
19     WHERE idCarrera = p_id;
20 END$$
21
22 CREATE PROCEDURE sp_listar_carreras()
23 BEGIN
24     SELECT idCarrera , nombreCarrera , descripcionCarrera
25     FROM Carrera;
26 END$$
27
28 CREATE PROCEDURE sp_actualizar_carrera(
29     IN p_id INT,
30     IN p_nombre VARCHAR(150),
31     IN p_descripcion VARCHAR(500)
32 )
33 BEGIN
34     UPDATE Carrera
35     SET nombreCarrera = p_nombre,
36         descripcionCarrera = p_descripcion
37     WHERE idCarrera = p_id;
38 END$$
39
40 CREATE PROCEDURE sp_eliminar_carrera(
41     IN p_id INT
42 )
43 BEGIN
44     DELETE FROM Carrera WHERE idCarrera = p_id;
45 END$$
46
47 DELIMITER ;

```

Operación	Método del DAO	Procedimiento	Sentencia interna
Alta	create	sp_crear_carrera(nombre, descripcion)	INSERT
Consulta por id	read	sp_leer_carrera(id)	SELECT ... WHERE
Consulta general	readAll	sp_listar_carreras()	SELECT
Cambio	update	sp_actualizar_carrera(id, nombre, descripcion)	UPDATE
Baja	delete	sp_eliminar_carrera(id)	DELETE

Tabla 1: Correspondencia entre las operaciones del CRUD, los métodos del DAO y los procedimientos

La figura 2 muestra que los cinco procedimientos existen en la base, consultados en information_schema. La figura 3 los ejecuta directamente desde el cliente de MySQL, sin pasar por Java: sp_listar_carreras y sp_leer_carrera(1) devuelven el registro que dejó la aplicación.

```
mysql> SELECT ROUTINE_NAME AS procedimiento, ROUTINE_TYPE AS tipo
-> FROM information_schema.ROUTINES
-> WHERE ROUTINE_SCHEMA = 'MisCursos' ORDER BY ROUTINE_NAME;
```

procedimiento	tipo
sp_actualizar_carrera	PROCEDURE
sp_crear_carrera	PROCEDURE
sp_eliminar_carrera	PROCEDURE
sp_leer_carrera	PROCEDURE
sp_listar_carreras	PROCEDURE

Figura 2: Los cinco procedimientos almacenados creados en la base MisCursos

```
mysql> CALL sp_listar_carreras();
```

idCarrera	nombreCarrera	descripcionCarrera
1	Sistemas Computacionales	Software, redes, bases de datos y nube

```
mysql> CALL sp_leer_carrera(1);
```

idCarrera	nombreCarrera	descripcionCarrera
1	Sistemas Computacionales	Software, redes, bases de datos y nube

Figura 3: Ejecución directa de dos procedimientos desde el cliente de MySQL

2.4. Los métodos CRUD con CallableStatement

Cada método del DAO abre una conexión, prepara la llamada con `prepareCall`, asigna los parámetros, ejecuta y cierra los recursos en un bloque `finally`. La sintaxis `{call nombre(?, ?)}` es la forma estándar de JDBC para invocar un procedimiento, independiente del manejador [4]. Los signos de interrogación son los parámetros, que se asignan por posición empezando en 1 (`setString(1, ...)`, `setInt(1, ...)`).

Código 3: Alta de una carrera: `create` llama a `sp_crear_carrera`

```
1 public void create(Carrera c) throws SQLException {
2     obtenerConexion();
3     CallableStatement cs = null;
4     try {
5         cs = conexion.prepareCall("{call sp_crear_carrera(?, ?)}");
6         cs.setString(1, c.getNombreCarrera());
7         cs.setString(2, c.getDescripcionCarrera());
8         cs.execute();
9     } finally {
10         if (cs != null) cs.close();
11         if (conexion != null) conexion.close();
12     }
13 }
```

`update` y `delete` siguen el mismo esquema, con `sp_actualizar_carrera` (tres parámetros: `id`, `nombre` y `descripcion`) y `sp_eliminar_carrera` (un parámetro: `id`). Los métodos de consulta cambian en un detalle: como sus procedimientos terminan en un `SELECT`, se ejecutan con `executeQuery()` y se recorre el `ResultSet`, en vez de `execute()`.

Código 4: Consulta general: `readAll` llama a `sp_listar_carreras`

```
1 public List<Carrera> readAll() throws SQLException {
```

```

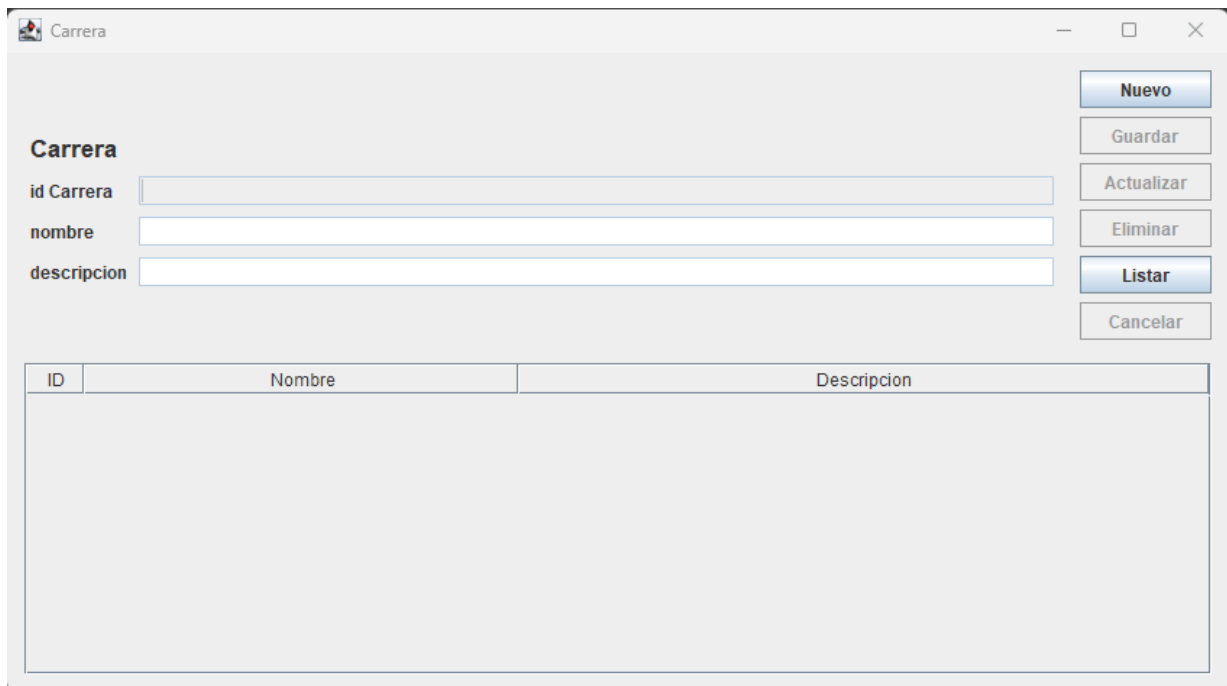
2     obtenerConexion();
3     CallableStatement cs = null;
4     ResultSet rs = null;
5     try {
6         cs = conexion.prepareCall("{ call sp_listar_carreras() }");
7         rs = cs.executeQuery();
8         return obtenerResultados(rs);
9     } finally {
10        if (rs != null) rs.close();
11        if (cs != null) cs.close();
12        if (conexion != null) conexion.close();
13    }
14 }

```

El método privado `obtenerResultados` convierte cada fila del `ResultSet` en un objeto `Carrera`, y `read` reutiliza ese mismo método y devuelve el primer elemento, o `null` si no hubo filas.

2.5. La interfaz gráfica y el CRUD completo

La ventana arranca con solo **Nuevo** y **Listar** habilitados y la tabla vacía (figura 4). Con **Nuevo** se habilitan el formulario y **Guardar** (figura 5). Al guardar, `create()` inserta el registro y la tabla se refresca leyendo de nuevo con `readAll()` (figuras 6 y 7).



The screenshot shows a window titled "Carrera". On the right side, there is a vertical stack of buttons: "Nuevo" (highlighted in blue), "Guardar", "Actualizar", "Eliminar", "Listar" (highlighted in blue), and "Cancelar". On the left side, under the heading "Carrera", there are three text input fields labeled "id Carrera", "nombre", and "descripcion". Below these fields is a table with three columns: "ID", "Nombre", and "Descripcion". The table is currently empty.

Figura 4: Estado inicial: solo Nuevo y Listar habilitados, tabla vacía

Carrera

id Carrera

nombre

descripcion

Buttons: Nuevo (disabled), Guardar (enabled), Actualizar (disabled), Eliminar (disabled), Listar (enabled), Cancelar (enabled).

ID	Nombre	Descripcion
----	--------	-------------

Figura 5: Alta: formulario lleno tras presionar Nuevo, con Guardar y Cancelar habilitados

Carrera

id Carrera

nombre

descripcion

Buttons: Nuevo (enabled), Guardar (disabled), Actualizar (disabled), Eliminar (disabled), Listar (enabled), Cancelar (disabled).

ID	Nombre	Descripcion
1	Sistemas Computacionales	Software, redes y bases de datos

Figura 6: Primera carrera guardada y leída de vuelta desde la base

The 'Carrera' window contains a form with three input fields: 'id Carrera', 'nombre', and 'descripcion'. To the right of the form are buttons: 'Nuevo', 'Guardar', 'Actualizar', 'Eliminar', 'Listar', and 'Cancelar'. Below the form is a table with the following data:

ID	Nombre	Descripcion
1	Sistemas Computacionales	Software, redes y bases de datos
2	Sistemas Electronicos	Electronica digital y analogica

Figura 7: Segunda alta: dos registros para probar cambio, baja y error

Intentar dar de alta un nombre que ya existe hace que MySQL rechace el INSERT dentro del procedimiento. La excepción sube por el DAO hasta la ventana, que la muestra en un cuadro de error (figura 8), y la tabla no cambia.

The 'Carrera' window is shown with the 'nombre' field filled with 'Sistemas Computacionales'. An error dialog box is overlaid on the window with the following text:

Error

No se pudo guardar la carrera:
Duplicate entry 'Sistemas Computacionales' for key 'Carrera.nombreCarrera'

Aceptar

Figura 8: Error al repetir un nombre: restricción UNIQUE de nombreCarrera

Al seleccionar una fila se habilitan **Actualizar** y **Eliminar** y el formulario se llena con sus datos; el id queda de solo lectura (figura 9). Tras editar la descripción y presionar **Actualizar**, `update()` llama a `sp_actualizar_carrera` y la tabla ya muestra el cambio (figura 10).

Carrera

id Carrera: 1

nombre: Sistemas Computacionales

descripcion: Software, redes, bases de datos y nube

Buttons: Nuevo, Guardar, Actualizar, Eliminar, Listar, Cancelar

ID	Nombre	Descripcion
1	Sistemas Computacionales	Software, redes y bases de datos
2	Sistemas Electronicos	Electronica digital y analogica

Figura 9: Carrera 1 seleccionada, con la descripción editada antes de Actualizar

Carrera

id Carrera:

nombre:

descripcion:

Buttons: Nuevo, Guardar, Actualizar, Eliminar, Listar, Cancelar

ID	Nombre	Descripcion
1	Sistemas Computacionales	Software, redes, bases de datos y nube
2	Sistemas Electronicos	Electronica digital y analogica

Figura 10: Carrera 1 con la descripción actualizada

Eliminar pide confirmación antes de llamar a `sp_eliminar_carrera` (figura 11). Una vez confirmado, la fila desaparece porque la tabla se vuelve a leer de la base (figura 12).

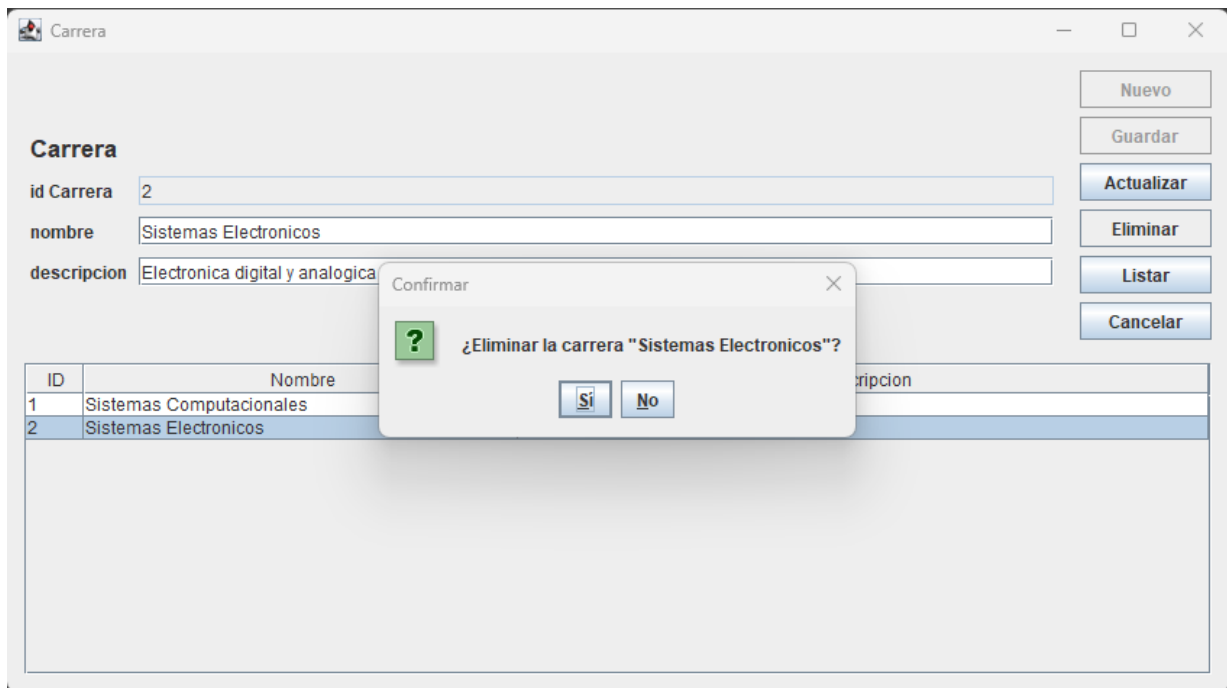


Figura 11: Confirmación antes de eliminar la carrera 2

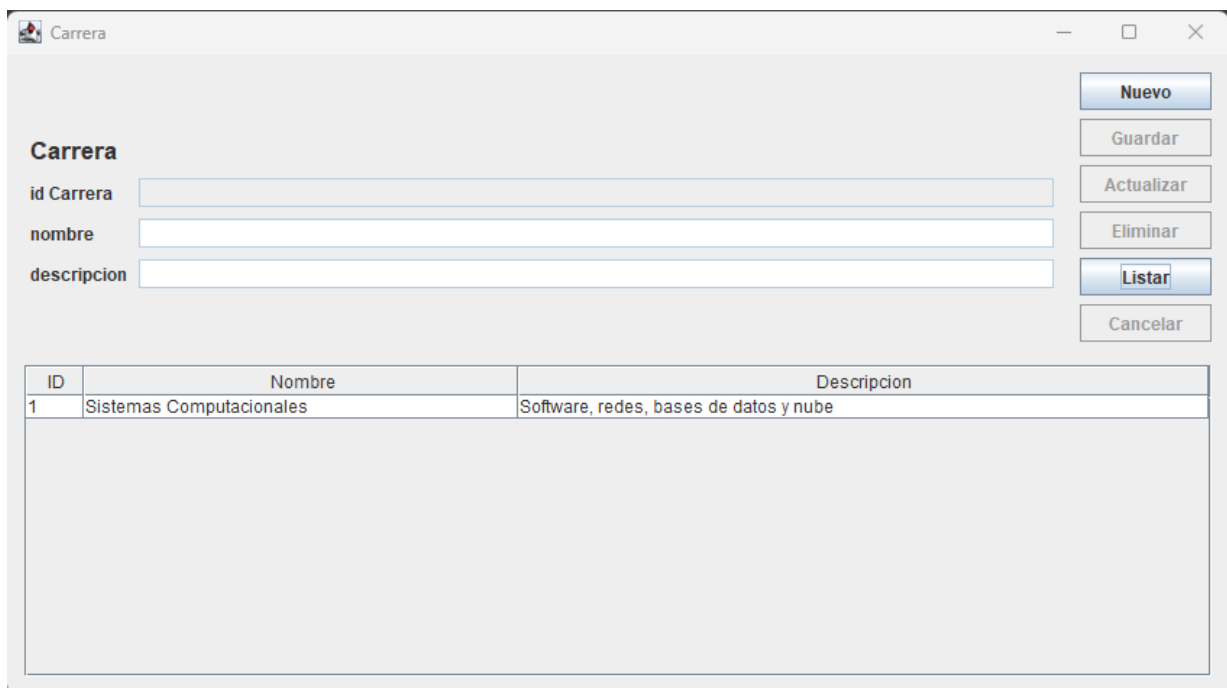


Figura 12: Estado final: solo queda la carrera 1, ya actualizada

2.6. Evidencia de la conexión JDBC y de las llamadas a los procedimientos

Para comprobar que la interfaz llega hasta la base, y que lo hace a través de los procedimientos, se activó temporalmente el registro general de consultas de MySQL (`general_log`) en el contenedor y se repitió toda la secuencia de las figuras anteriores. La figura 13 muestra el resultado. La primera consulta lista las conexiones que no vienen del propio contenedor: son las del DAO, que entran por TCP desde el equipo anfitrión (172.19.0.1) a la base `MisCursos`. Aparece más de una porque cada método del DAO abre y cierra su propia conexión en `obtenerConexion()`.

La segunda consulta lista lo que el servidor recibió con `CALL`. Cada acción de la ventana llegó como una llamada a un procedimiento: `sp_crear_carrera` en las altas, `sp_actualizar_carrera` en el cambio y `sp_eliminar_carrera` en la baja, cada una seguida de `sp_listar_carreras` para refrescar la tabla. La tercera

llamada a `sp_crear_carrera`, con el nombre repetido, es el intento de la figura 8: llegó al servidor pero no insertó nada. Las horas están en la zona horaria del contenedor (UTC). Al terminar se desactivó el registro y se vació.

```
mysql> SELECT DATE_FORMAT(event_time,'%H:%i:%s') AS hora,
->      CONVERT(argument USING utf8mb4) AS conexion
-> FROM mysql.general_log
-> WHERE command_type='Connect' AND user_host NOT LIKE '%localhost%'
-> ORDER BY event_time LIMIT 2;
+-----+-----+
| hora   | conexion                                     |
+-----+-----+
| 23:16:25 | root@172.19.0.1 on MisCursos using SSL/TLS |
| 23:16:28 | root@172.19.0.1 on MisCursos using SSL/TLS |
+-----+-----+

mysql> SELECT DATE_FORMAT(event_time,'%H:%i:%s') AS hora,
->      CONVERT(argument USING utf8mb4) AS llamada
-> FROM mysql.general_log
-> WHERE argument LIKE 'CALL sp%' ORDER BY event_time;
+-----+-----+
| hora   | llamada                                     |
+-----+-----+
| 23:16:25 | CALL sp_listar_carreras()                  |
| 23:16:28 | CALL sp_crear_carrera('Sistemas Computacionales', 'Software, redes y bases de datos') |
| 23:16:28 | CALL sp_listar_carreras()                  |
| 23:16:28 | CALL sp_crear_carrera('Sistemas Electronicos', 'Electronica digital y analogica') |
| 23:16:28 | CALL sp_listar_carreras()                  |
| 23:16:29 | CALL sp_crear_carrera('Sistemas Computacionales', 'Otra descripcion') |
| 23:16:32 | CALL sp_actualizar_carrera(1, 'Sistemas Computacionales', 'Software, redes, bases de datos y nube') |
| 23:16:32 | CALL sp_listar_carreras()                  |
| 23:16:34 | CALL sp_eliminar_carrera(2)                |
| 23:16:34 | CALL sp_listar_carreras()                  |
+-----+-----+
```

Figura 13: Registro del servidor: conexiones JDBC del DAO y llamadas a los procedimientos

Por último, la figura 14 consulta la tabla directamente. El único registro y su descripción coinciden con el estado final de la ventana (figura 12).

```
mysql> SELECT * FROM Carrera;
+-----+-----+-----+
| idCarrera | nombreCarrera          | descripcionCarrera          |
+-----+-----+-----+
|          1 | Sistemas Computacionales | Software, redes, bases de datos y nube |
+-----+-----+-----+
```

Figura 14: Contenido real de la tabla tras las operaciones hechas desde la ventana

3. Conclusiones

Tipos de *statement*. JDBC ofrece tres: `Statement` (SQL como texto, sin parámetros), `PreparedStatement` (SQL con parámetros `?`, precompilado) y `CallableStatement`, que hereda del anterior y sirve para invocar procedimientos almacenados. Con `CallableStatement` el texto de la consulta en Java se reduce a `{call sp_x(?, ?)}`: el SQL vive en el servidor y se puede cambiar sin recompilar la aplicación, aunque hay que instalar los procedimientos junto con la base. En esta práctica, la ventana y `Carrera` solo dependen de los cinco métodos del DAO, así que todo lo relacionado con los procedimientos queda contenido en `CarreraDAO`.

Tipos de parámetros y de datos. Un procedimiento puede declarar parámetros `IN`, `OUT` e `INOUT`. Los cinco de esta práctica son solo `IN`, así que basta con `setInt` y `setString`. Los parámetros `OUT` requieren registrarse antes de ejecutar con `registerOutParameter` [5]. El tipo del `set` debe corresponder al de la columna (`INT` con `setInt`, `VARCHAR` con `setString`).

Errores más típicos. Casi todos llegan como `SQLException` o una de sus subclases. La tabla 2 resume los que se pueden presentar con esta configuración. De ellos, solo el de nombre duplicado se reprodujo en la

práctica (figura 8); los demás se describen según la documentación y por cómo está armado el proyecto, sin haberse provocado.

Error	Causa típica	Cómo se evita o se resuelve
Nombre duplicado (<code>SQLIntegrityConstraint ViolationException</code>)	Se repite <code>nombreCarrera</code> , que es <code>UNIQUE</code> . Es el que se vio en la figura 8.	Capturar la excepción y avisar al usuario, como hace la ventana.
Conexión rechazada o <i>Access denied</i>	Contenedor apagado, puerto equivocado, o usuario o contraseña que no coinciden con los del contenedor.	Revisar que <code>docker ps</code> muestre el contenedor y que la URL, el usuario y la clave del DAO coincidan con el <code>docker-compose.yml</code> .
Puerto equivocado	Este contenedor publica MySQL en el puerto 3307, no en el 3306 que es el habitual. Con la URL apuntando al 3306, la conexión se rechaza o llega a otro servidor MySQL del equipo.	Confirmar el puerto en la URL antes de ejecutar.
El procedimiento no existe	Los scripts de <code>init/</code> solo corren cuando el contenedor se crea desde cero. Si la base ya existía, no se crean los procedimientos.	Ejecutar <code>02-procedimientos.sql</code> a mano, o recrear el contenedor.
Número o tipo de parámetros incorrecto	Llamar <code>{call sp_crear_carrera(?)}</code> con dos valores, o usar <code>setInt</code> donde va <code>setString</code> . Los índices empiezan en 1, no en 0.	Que la firma de <code>prepareCall</code> coincida con la del procedimiento (tabla 1).
<code>execute</code> contra <code>executeQuery</code>	Usar <code>executeUpdate</code> con un procedimiento que devuelve filas, o <code>executeQuery</code> con uno que no devuelve nada.	<code>executeQuery</code> para <code>sp_leer_carrera</code> y <code>sp_listar_carreras</code> ; <code>execute</code> para los otros tres.
Falta <code>DELIMITER</code> al crear procedimientos	Sin cambiar el delimitador, el cliente corta el script en el primer <code>;</code> del cuerpo y el <code>CREATE PROCEDURE</code> queda incompleto.	Envolver el script con <code>DELIMITER \$\$... DELIMITER ;</code> .
<code>ClassNotFoundException</code>	El conector de MySQL no está en el <i>classpath</i> .	Incluir el <code>.jar</code> al compilar y al ejecutar; el DAO lo convierte en <code>SQLException</code> .
Recursos sin cerrar	Conexiones y <code>CallableStatement</code> abiertos si ocurre una excepción antes del <code>close()</code> .	Cerrarlos en <code>finally</code> , como hacen los cinco métodos.

Tabla 2: Errores típicos al trabajar con JDBC, DAO y procedimientos almacenados

El DAO conserva su estructura; lo que envía ahora es una llamada en lugar de una consulta. El costo es que el nombre y los parámetros del procedimiento en la base deben coincidir con la cadena `{call ...}` en Java.

4. Referencias Bibliográficas

- [1] Deepak Alur, John Crupi y Dan Malks. *Core J2EE Patterns: Best Practices and Design Strategies*. 2.^a ed. Prentice Hall, 2003.
- [2] Oracle Corporation. *MySQL Connector/J Developer Guide: Overview of MySQL Connector/J*. URL: <https://dev.mysql.com/doc/connector-j/en/connector-j-overview.html> (visitado 19-09-2026).
- [3] Oracle Corporation. *MySQL 8.0 Reference Manual: CREATE PROCEDURE and CREATE FUNCTION Statements*. URL: <https://dev.mysql.com/doc/refman/8.0/en/create-procedure.html> (visitado 19-09-2026).
- [4] Oracle Corporation. *CallableStatement (Java SE 21 & JDK 21 API)*. URL: <https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/CallableStatement.html> (visitado 19-09-2026).
- [5] Oracle Corporation. *MySQL Connector/J Developer Guide: Using JDBC CallableStatements to Execute Stored Procedures*. URL: <https://dev.mysql.com/doc/connector-j/en/connector-j-usagenotes-statements-callable.html> (visitado 19-09-2026).
- [6] Oracle Corporation. *Lesson: JDBC Introduction (The Java Tutorials)*. URL: <https://docs.oracle.com/javase/tutorial/jdbc/overview/index.html> (visitado 19-09-2026).